DAY 03 · LAB

Living in the **Cloud**.

DURATION

90 minutes

STRUCTURE

16 steps, 5 parts

YOU NEED

A free Cloudflare account

PAYMENT DETAILS

None required

Publish a page in eight minutes. Publish it again with no page at all. Then work out **what it should have cost**.

You will build the same thing twice, at two different levels of abstraction. Then you will climb the whole ladder, from owning a machine to owning nothing, and count what you gave away at each rung.

Finally you will go back to the facility you sized on Monday and price it as a cloud bill. That answer surprises most people, including people who buy cloud for a living.

PART	STEPS	TIME
Setup	**1**	5 min
Ship a page	**2 to 4**	15 min
The same page, generated	**5 to 8**	20 min
The abstraction ladder	**9 to 11**	15 min
Rent or own	**12 to 15**	25 min
Findings	**16**	**10 min**

01

Confirm you can get in

You need two things and neither costs anything.

- A free account at `dash.cloudflare.com/sign-up`, with the verification email confirmed
- A plain text editor. Notepad, TextEdit in plain text mode, VS Code, anything that saves a `.html` file without adding formatting

> If signup is rate limited from the campus network, use mobile data, or fall back to **app.netlify.com/drop**, which takes a dragged folder with no account at all. Everything after Part 1 still works.

WHAT YOU ARE ABOUT TO NOTICE

On Monday, putting a server on the internet required a building, a substation, two years and USD 2.2 million.

This afternoon it requires a browser tab and about eight minutes.

> Both of those statements are true at the same time. Part 4 is where you work out why anybody still chooses the first one.

02

Make the file

Create a folder called `mysite`, and this file inside it as `index.html`. Change the name and the city.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>A page that exists</title>
  <style>
    body { margin:0; min-height:100vh; display:grid; place-items:center;
           background:#0d1b33; color:#eef2f8;
           font-family: system-ui, sans-serif; }
    main { max-width:40rem; padding:2rem; }
    h1   { font-weight:300; font-size:3rem; margin:0 0 1.5rem; }
  </style>
</head>
<body>
  <main>
    <h1>This page was written once and copied everywhere.</h1>
    <p>Published by YOUR NAME, from YOUR CITY.</p>
    <p>Nobody ran any code to produce what you are reading. A machine
       found a file on a disk and sent it to you unchanged.</p>
  </main>
</body>
</html>
```

03

Publish it

In the dashboard: **Workers & Pages**, **Create**, the **Pages** tab, then **Upload assets**.

Name the project, drag the folder in, deploy.

> You get a URL ending `.pages.dev`. Open it on your phone. It is live, worldwide, on someone else's network, and you have paid **nothing**.

The dashboard's wording moves around. If a label does not match, look for the words Pages and upload. The concept is stable even when the buttons are not.

STEP 04 · THREE QUESTIONS. WRITE ONE LINE EACH BEFORE MOVING ON

4A

How many servers did you configure, patch, or choose the location of?

On Monday the same question took a team of three an entire afternoon.

4B

Who obtained the certificate that put the padlock in your address bar, and when did you ask them to?

A decade ago this was a purchase, a support ticket, and an annual renewal that people forgot.

4C

One million people load your page in the next sixty seconds. What breaks?

Be specific about which part of the system you think is under pressure, and why.

05

Create the Worker, then replace the code

Workers & Pages, **Create**, the **Workers** tab, start from Hello World, deploy the default, then open the editor and paste this.

```
export default {
  async fetch(request) {
    const cf  = request.cf || {};
    const now = new Date().toISOString().replace("T", " ").slice(0, 19);

    const html = `<!doctype html>
<html><head><meta charset="utf-8"><title>A page that did not exist</title>
<style>
  body { margin:0; min-height:100vh; display:grid; place-items:center;
         background:#0d1b33; color:#eef2f8; font-family:system-ui,sans-serif; }
  main { max-width:40rem; padding:2rem; }
  h1 { font-weight:300; font-size:3rem; margin:0 0 1.5rem; }
  dt { text-transform:uppercase; letter-spacing:.18em; font-size:.7rem; opacity:.55; }
  dd { margin:0 0 1.25rem; font-size:1.2rem; }
</style></head>
<body><main>
  <h1>This page did not exist until you asked for it.</h1>
  <dl>
    <dt>Generated at</dt><dd>${now} UTC</dd>
    <dt>You appear to be in</dt><dd>${cf.country || "unknown"}</dd>
    <dt>Served from</dt><dd>${cf.colo || "unknown"}</dd>
    <dt>Your network</dt><dd>${cf.asOrganization || "unknown"}</dd>
  </dl>
</main></body></html>`;

    return new Response(html, {
      headers: { "content-type": "text/html;charset=UTF-8",
                 "cache-control": "no-store" },
    });
  },
};
```

06

Deploy and reload it

Open the `.workers.dev` URL and reload several times. Watch the timestamp change.

> The three letter code next to "Served from" is an airport code, the same kind you decoded on Tuesday. It is the location that **ran your code**. The country was not sent by you. The network worked it out from your address before your code ever ran.

Important. `request.cf` is only filled in on the deployed URL. In the dashboard preview it shows as unknown. That is expected, not a bug in your code.

07

Make it yours

Change something so the page differs per visitor. Any of these.

- Greet visitors from your own country differently from everyone else
- Show a different colour before and after 18:00 UTC
- Count down to the last day of the course
- Refuse anyone whose network organisation contains a word you dislike, then put it back, having briefly understood what a firewall is

08

Break it on purpose

Introduce a deliberate error. Delete a closing brace, or change `Response` to `Respons`. Deploy it.

Load the page. Then undo it and deploy again.

> Note how long the broken version was live, and who could see it. **Not some of your visitors. All of them, everywhere, at the same instant.**

On Monday, a mistake affected one building. Here, a one character error is worldwide before you have finished reading the confirmation message. That is the trade you accepted when you stopped choosing servers.

STEP 09 · 15 MINUTES · FILL IN WHO DECIDES. WRITE "YOU" OR "THEM" IN EVERY EMPTY CELL

DECISION	OWN THE MACHINE	RENT A MACHINE	PAGES	WORKER	A HOSTED SERVICE
Choose the building	you	them			
Choose the machine	you	you			
Choose the operating system	you	you			
Patch the operating system					
Decide how it scales					
Obtain the certificate					
Decide which region it runs in					
Write the code					
Count of "you"					

THEN READ THE BOTTOM ROW LEFT TO RIGHT

That row has a name. Roughly, the columns are infrastructure as a service, platform as a service, functions, and software as a service. **Every step to the right is a decision you stopped making, and a decision you can no longer make.**

10

Place these on the ladder

Which rung is each one? There is a defensible answer for each, and one of them is genuinely arguable.

- A virtual machine you log into and install things on
- A managed database that you never patch
- Gmail
- The Worker you built twenty minutes ago
- The static page you built forty minutes ago
- A container platform your team operates itself

11

The question that matters

You went four rungs up the ladder this afternoon and it felt like pure gain. Name three things you can no longer do.

One

Two

Three

> Hints if you are stuck: what happens if you need software that is not JavaScript, if a job takes four minutes, if you need the data to stay in one country, or if you want to move to a different company next year?

25 MINUTES · BACK TO MONDAY'S FACILITY, PRICED AS A CLOUD BILL

12

The rate card

Rounded published rates, for this exercise. Real prices vary by provider, region and negotiation.

A cloud machine like one of Monday's servers	**USD 1.00 per hour**
The same machine, three year commitment	**USD 0.40 per hour**
Getting data out of the cloud	**USD 0.09 per GB**
Hours in a year	**8,760**

13

Price Monday's facility three ways

400 servers. Monday's build cost was USD 2.2 million, and the electricity was USD 289,000 a year.

Cloud, on demand, all year	400 × 1.00 × 8,760
Cloud, three year commitment	400 × 0.40 × 8,760
Own it, per year	BUILD ÷ 5 + ELECTRICITY

Spread the building over five years. That is roughly how long the servers last before they need replacing anyway.

14

Find the crossover

Owning costs the same whether the machines are busy or idle. Renting only costs money while they are running.

So: what fraction of the year would you have to use these machines before renting becomes the cheaper option?

```
owning, per year        ______
renting, on demand      ______

crossover = owning / renting = ______  =  ______ %
```

> Sanity check: your answer should be well under half. If it is over 50%, check which number you divided by which.

15

The honest column

Both of your numbers are lies of omission. List what each one leaves out.

OWNING LEAVES OUT	RENTING LEAVES OUT

> Prompts: who works there? What happens in year six? What did the two year wait cost you? And on the other side, what did you pay to get your data back out?

10 MINUTES · IN WRITING

01 Your static page and your Worker both answer instantly. Explain the difference to somebody who has not done this lab, in two sentences, without using the word serverless.

02 You climbed four rungs of the ladder. At which rung would you stop for a university's student records system, and why that rung rather than the one above?

03 Given your crossover number, name one workload that clearly should be rented and one that clearly should be owned. Say which number in your working decided it.

04 Monday's facility took two years and a substation. This afternoon took eight minutes and no permission from anybody. **Name one thing that is worse about the eight minute version**, and be specific.

ADD AN API, THEN WATCH A BROWSER REFUSE IT

```
const url = new URL(request.url);

if (url.pathname === "/api/time") {
  return new Response(
    JSON.stringify({ now: new Date().toISOString(),
                     colo: cf.colo }),
    { headers: {
        "content-type": "application/json",
        "access-control-allow-origin": "*" } }
  );
}
```

Call it from your static page with `fetch()`. Then delete that last header, redeploy, and reload.

It stops working and the console tells you why. You have met the rule that stops any page you visit from quietly reading your webmail.

REGION SHOPPING

Pick any large provider and open its region list. Count them. Note which countries have several and which continents have almost none.

Now pick a region for a hypothetical health records system serving your country, and write one sentence justifying it on grounds that are **not** latency.

THE EGRESS TRAP

Your facility stores 12 PB. At USD 0.09 per GB, what does it cost to take all of it back out of a cloud provider, once?

---

1 PB is roughly 1,000,000 GB. The answer is why people talk about data having gravity.

HAND IN, AND KEEP BOTH URLS FOR TOMORROW

There is a server. There are hundreds of thousands of **them**.

ONE

Two live URLs, one a file and one a decision

TWO

The completed ladder table and your count row

THREE

Three prices and your crossover percentage

FOUR

The four answers from Step 16

SYMPTOM	CAUSE	FIX
Country and location show unknown	You are on the dashboard preview	Open the deployed workers.dev URL
The browser shows raw HTML as text	Missing or misspelt content type header	It must read `text/html`, exactly
A syntax error you cannot see	A backtick in your page text closed the template early	Remove backticks from the visible text
Changes do not appear	Cached response, or an edit you did not deploy	Hard reload, and confirm you pressed deploy
No workers.dev subdomain	Not yet chosen on the account	The dashboard prompts once. Pick anything
Deployment works, the URL 404s	**You uploaded the folder rather than its contents**	**Re-upload so index.html sits at the root**

Free tier is 100,000 requests a day. Do not run a load test on it. Tomorrow you will send a few hundred, well inside the limit.